

Embedded Linux & Yocto (*Week 1*)

Raspberry Pi Zero 2W Setup
Shell Package Manager
TCP Server/Client

By: Talel BELHAJ SALEM

Setup

Raspberry Pi Imager

- Go to <https://www.raspberrypi.com/software>
- Download Raspberry Pi Imager

Raspberry Pi Imager

Raspberry Pi Imager is the quick and easy way to install **Raspberry Pi OS** and other operating systems to a microSD card, ready to use with your Raspberry Pi.

Download and install Raspberry Pi Imager on a computer with an SD card reader. Insert the microSD card you'll use with your Raspberry Pi into the reader and run Raspberry Pi Imager.

[Download for Debian or Ubuntu \(x86_64\)](#)

To install on **Raspberry Pi OS**, type
`sudo apt install rpi-imager`
into a terminal window

[Download for Windows](#)

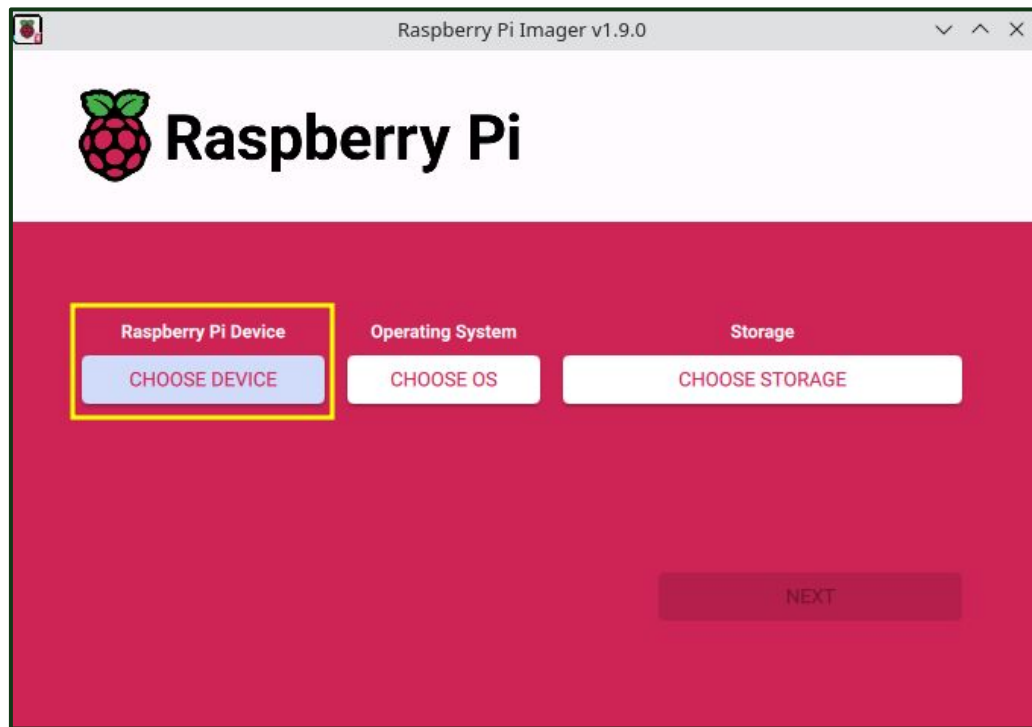
[Download for macOS](#)

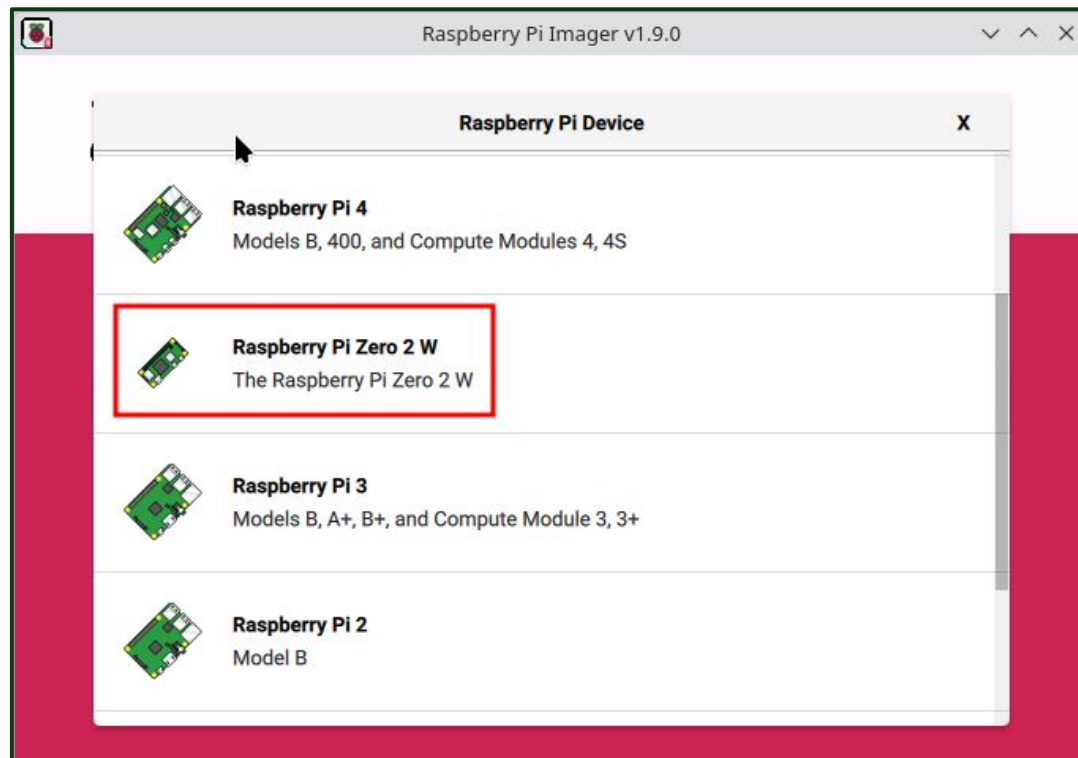


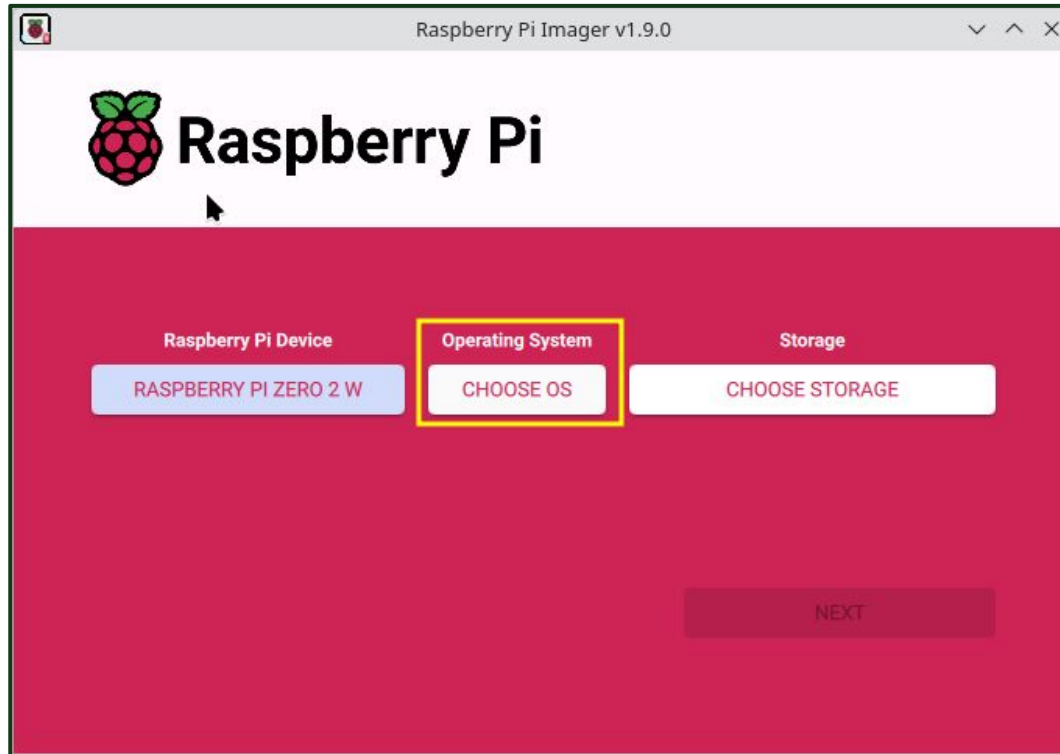
- If you download windows version, just open the executable and follow instructions
- If you download the **.deb** version, install it with:

```
#> sudo dpkg -i ~/Downloads/rpi-imager-*.deb
```

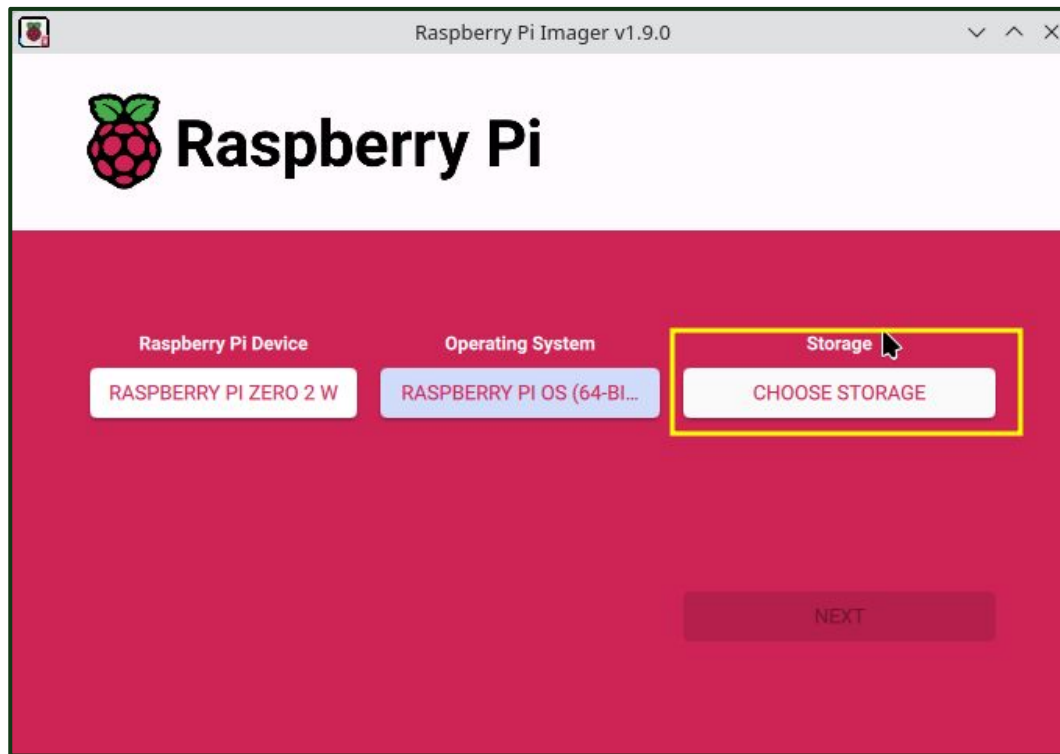
Flashing

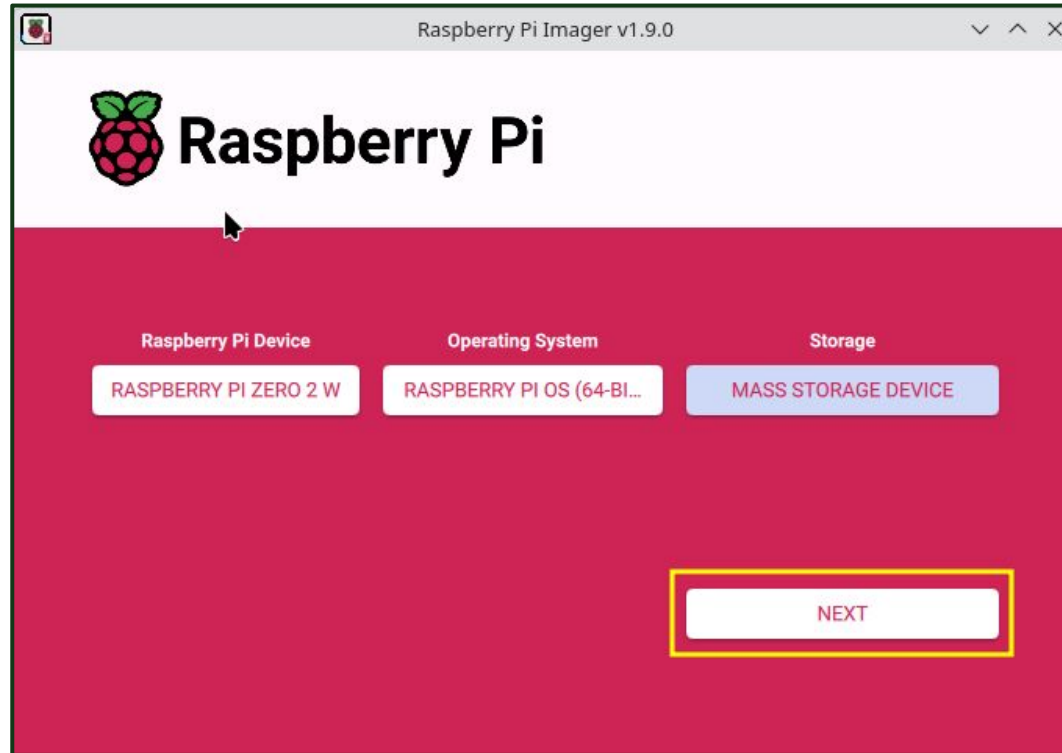


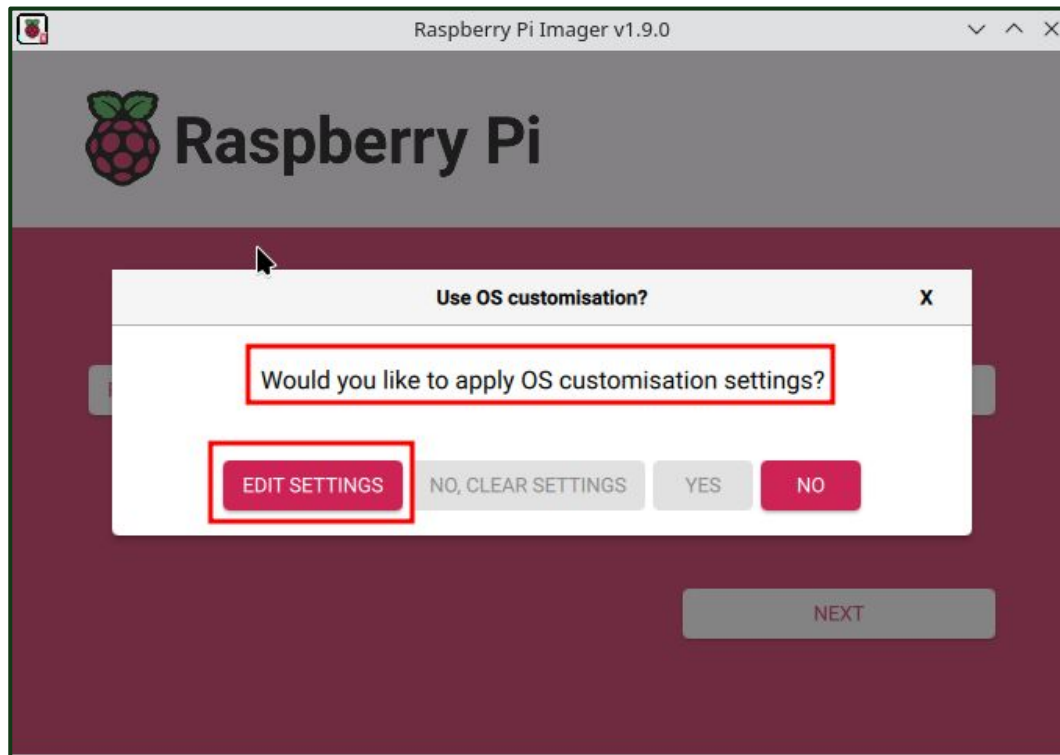




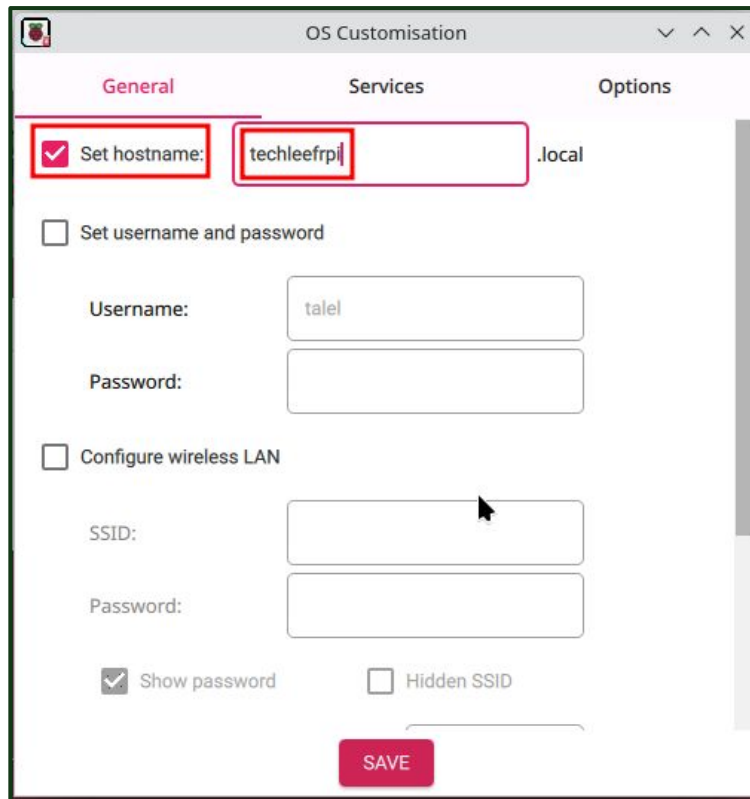








- Set any name you want

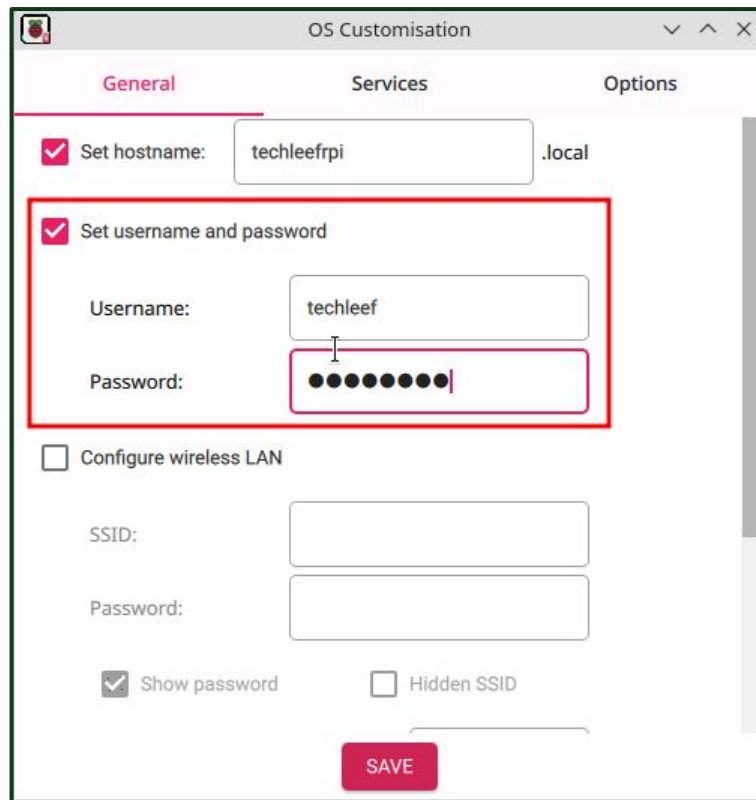


The screenshot shows a window titled "OS Customisation" with three tabs: "General", "Services", and "Options". The "General" tab is active. It contains the following options:

- ☒ Set hostname: .local
- ☐ Set username and password
 - Username:
 - Password:
- ☐ Configure wireless LAN
 - SSID:
 - Password:
 - ☒ Show password ☐ Hidden SSID

A red box highlights the "Set hostname:" checkbox and the hostname input field. A pink box highlights the hostname input field. A pink button labeled "SAVE" is at the bottom.

- Define your own password

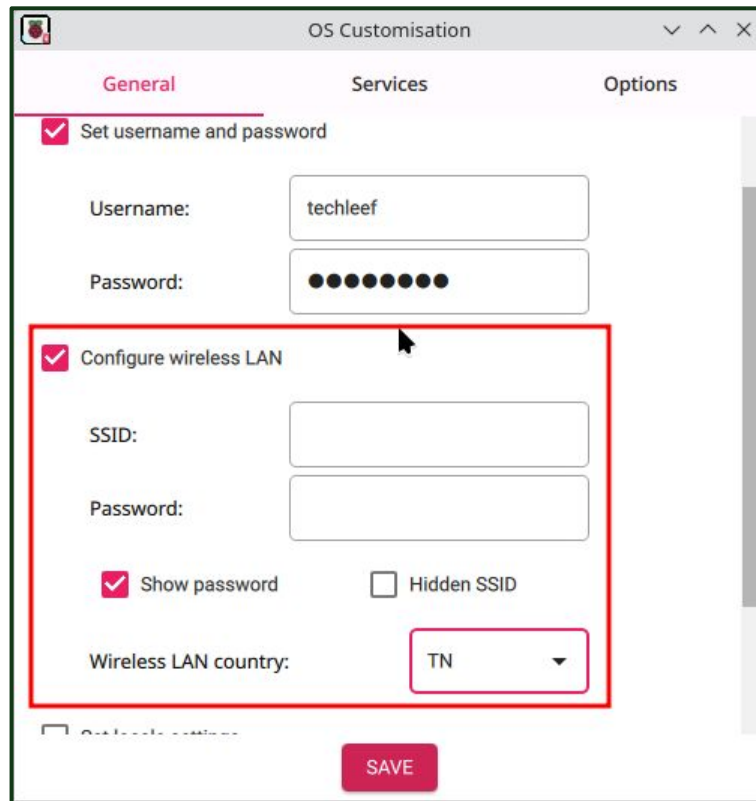


The screenshot shows a window titled "OS Customisation" with three tabs: "General", "Services", and "Options". The "General" tab is active. It contains the following fields and options:

- ☒ Set hostname: techleefrpi.local
- ☒ Set username and password
 - Username: techleef
 - Password: [masked with 12 dots]
- ☐ Configure wireless LAN
 - SSID: [empty field]
 - Password: [empty field]
 - ☒ Show password
 - ☐ Hidden SSID

A red rectangular box highlights the "Set username and password" section. A pink rectangular box highlights the password input field. A pink "SAVE" button is located at the bottom center of the window.

- WiFi is optional

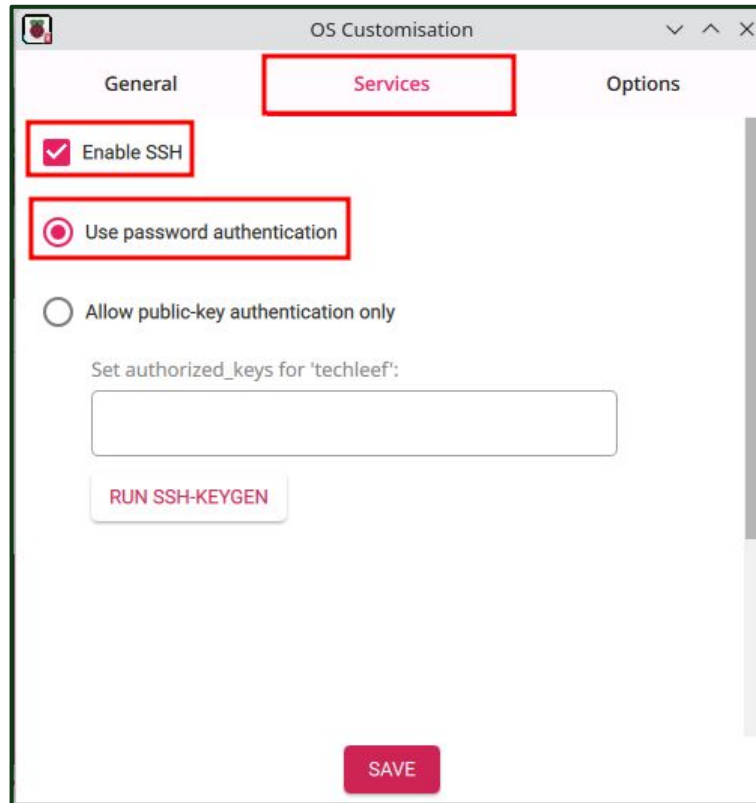


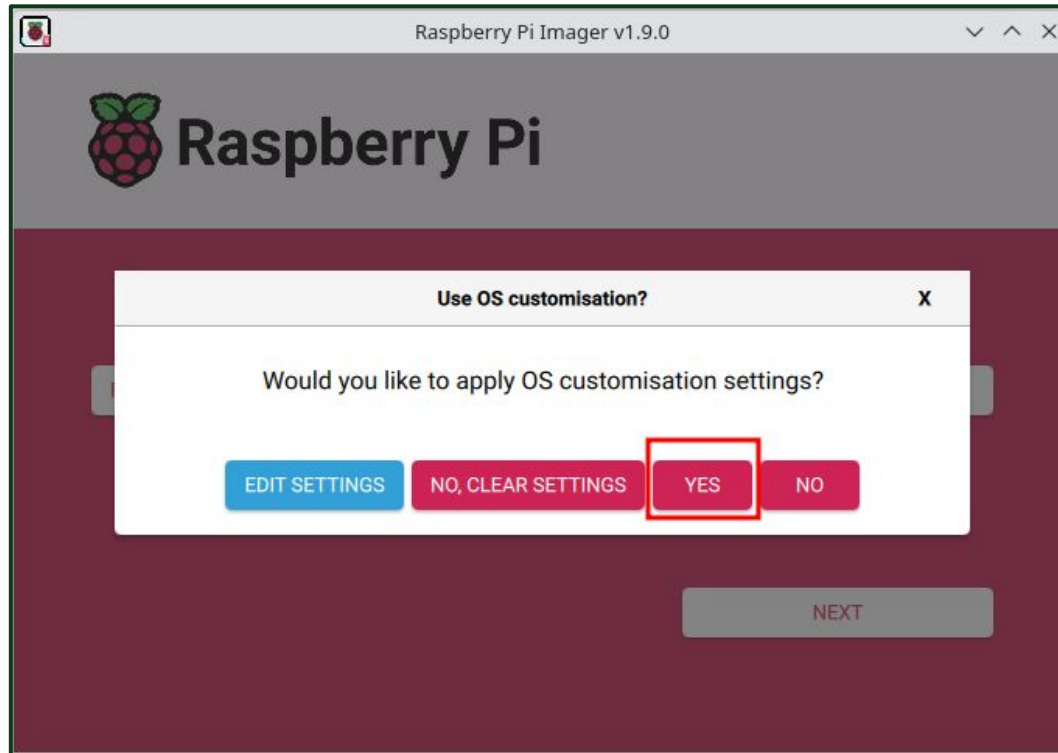
The screenshot shows a window titled "OS Customisation" with three tabs: "General", "Services", and "Options". The "General" tab is active. It contains the following options:

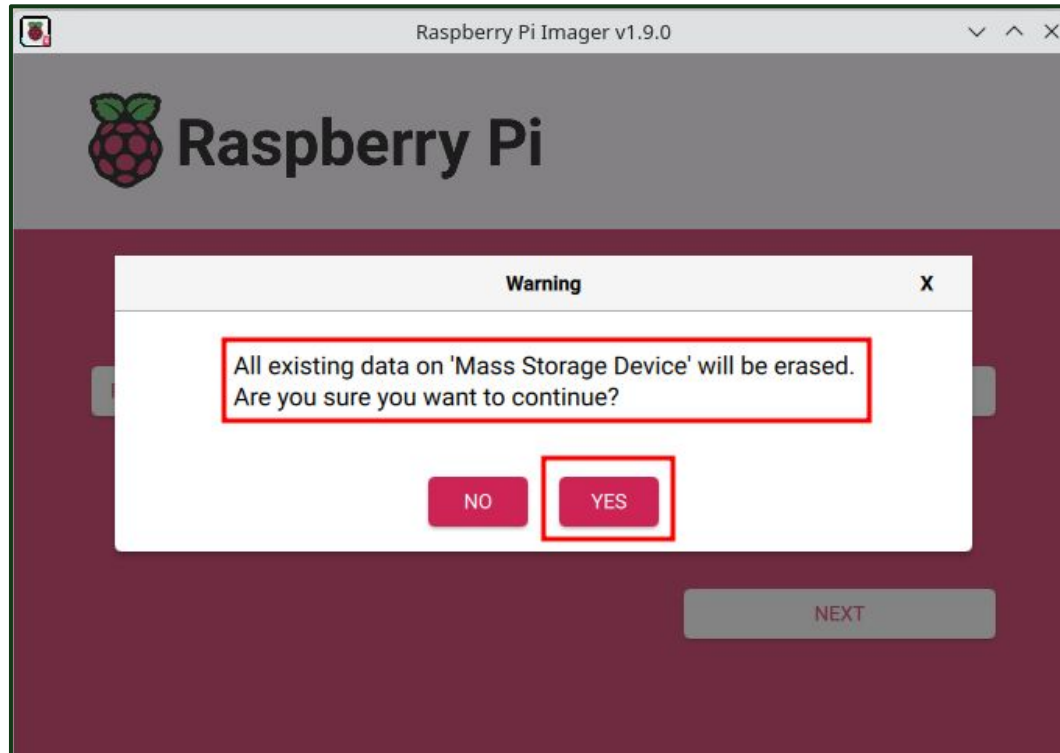
- ☒ Set username and password
 - Username: techleef
 - Password: [masked]
- ☒ Configure wireless LAN (highlighted with a red box)
 - SSID: [empty field]
 - Password: [empty field]
 - ☒ Show password ☐ Hidden SSID
 - Wireless LAN country: TN (dropdown menu)
- ☐ Set network settings

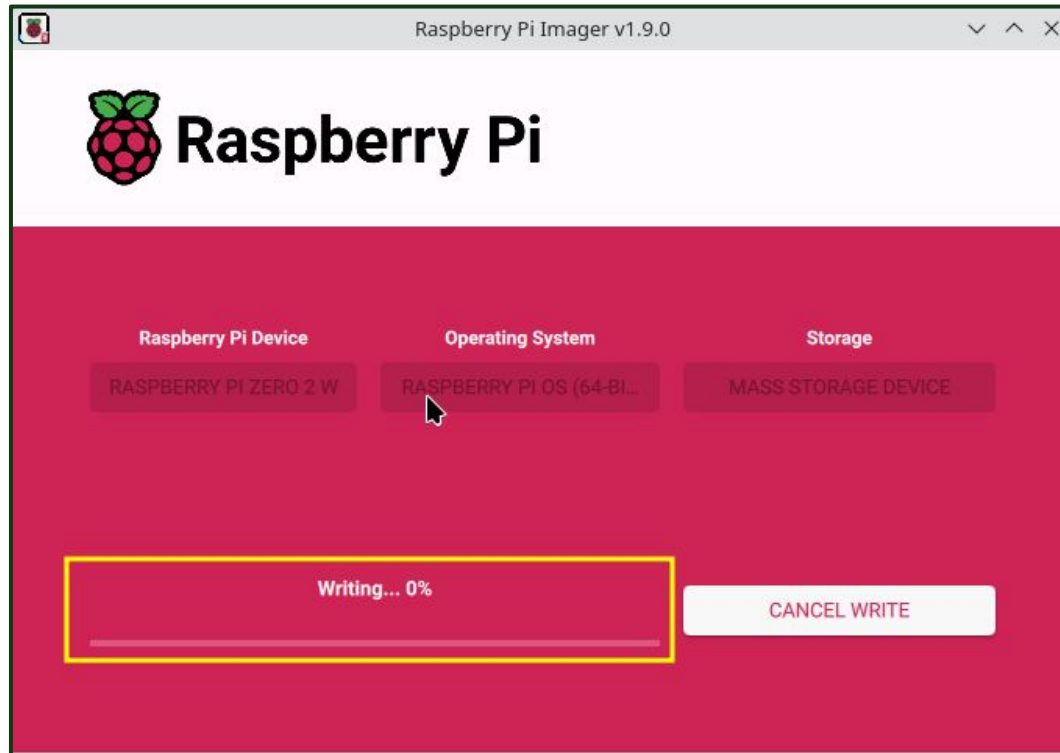
A "SAVE" button is located at the bottom right of the window.

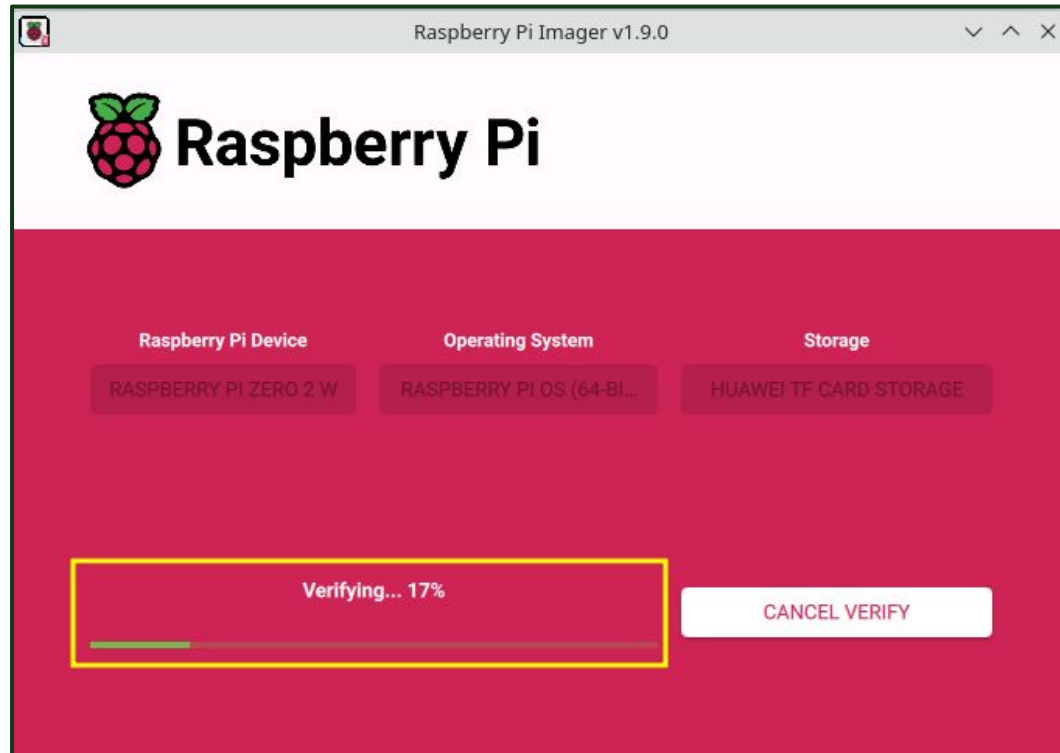
- Enable SSH
- Then click **save**

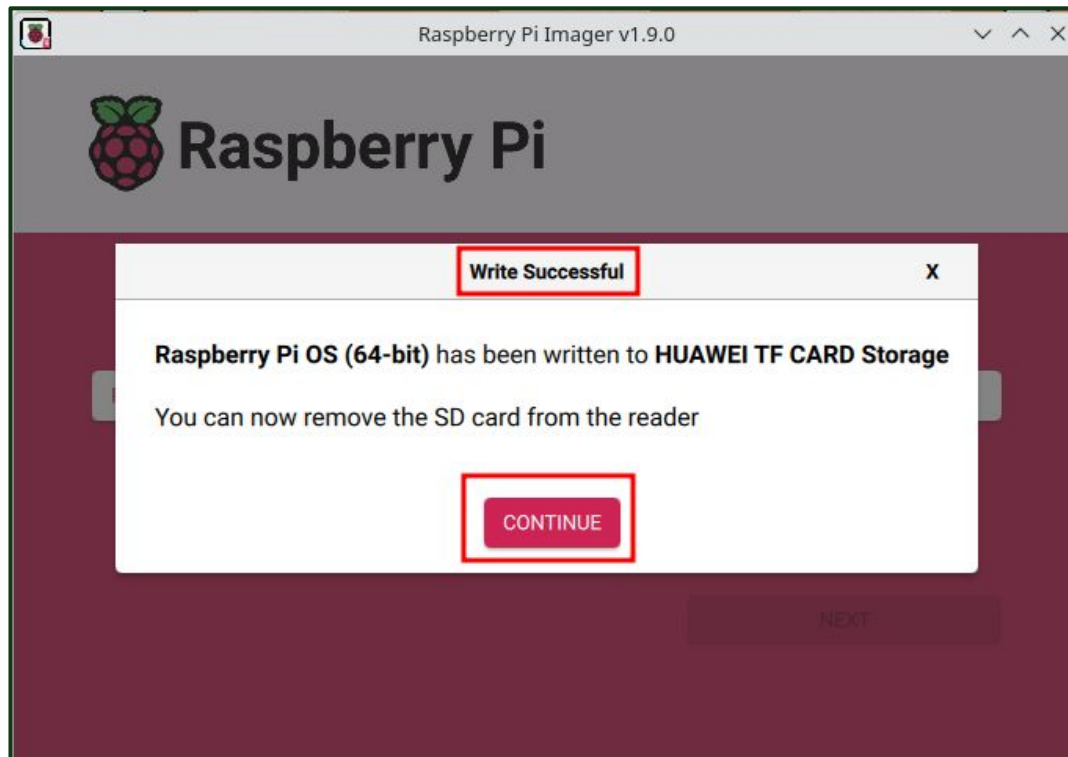












Enable UART

- By default, UART is not enabled, so we need to enable it before unplugging the sdcard
- When you plug the sdcard in your PC, check the dmesg output to see that the sdcard is detected

```
[87492.555754] scsi 0:0:0:0: Direct-Access      HUAWEI    TF CARD Storage  2.31 PQ: 0 ANSI: 2
[87492.555958] sd 0:0:0:0: Attached scsi generic sg0 type 0
[87492.557432] sd 0:0:0:0: [sda] 30535680 512-byte logical blocks: (15.6 GB/14.6 GiB)
[87492.557590] sd 0:0:0:0: [sda] Write Protect is off
[87492.557593] sd 0:0:0:0: [sda] Mode Sense: 0f 00 00 00
[87492.557872] sd 0:0:0:0: [sda] Write cache: enabled, read cache: enabled, ..
[87492.601305] sda: sda1 sda2
[87492.601562] sd 0:0:0:0: [sda] Attached SCSI removable disk
```


- Let's check the list of our block devices, there we should find **sda1** (boot) and **sda2** (rootfs)
- Let's mount the boot partition where we will find the configuration files of the board

```
#> lsblk
```

NAME	MAJ:MIN	RM	SIZE	RO	TYPE	MOUNTPOINTS
sda	8:0	1	14.6G	0	disk	
└─sda1	8:1	1	512M	0	part	
└─sda2	8:2	1	5.2G	0	part	

```
#> sudo mkdir /mnt/bootfs
```

```
#> sudo mount /dev/sda1 /mnt/bootfs
```

```
#> mountpoint /mnt/bootfs  
/mnt/bootfs/ is a mountpoint
```

- We use “**tee**” to append a text from the echo command into the **config.txt** file
- The “**tee**” command prints whatever it writes to the file, so we redirect that to **/dev/null** as we don’t need it
- After that, we unmount the partition, unplug the sdcard and insert it in the board

```
#> cd /mnt/bootfs
#> ls *.txt

cmdline.txt  config.txt  issue.txt

#> echo “enable_uart=1” | sudo tee -a config.txt > /dev/null

#> grep uart config.txt

enable_uart=1

#> sudo umount /mnt/bootfs
```

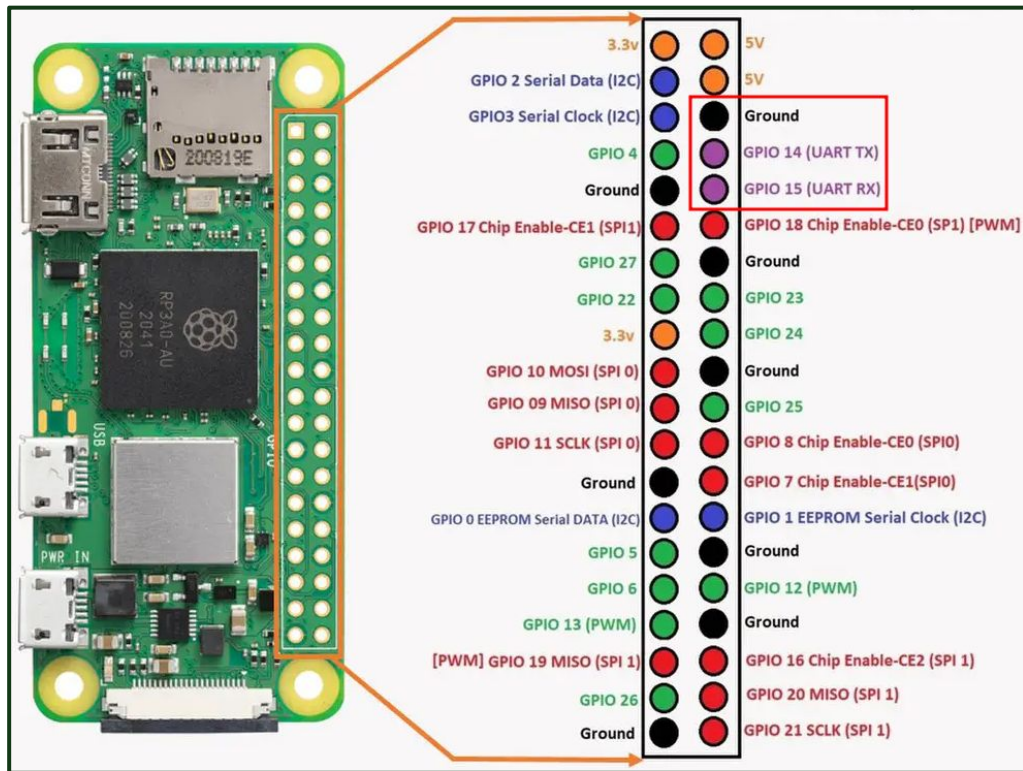
Booting

- Let's plug the **FTDI** module and check `dmesg` to see the USB device created under **/dev**

```
[89747.822013] usb 1-1: new full-speed USB device number 34 using xhci_hcd
[89747.946576] usb 1-1: New USB device found, idVendor=10c4, idProduct=ea60, bcdDevice= 1.00
[89747.946581] usb 1-1: New USB device strings: Mfr=1, Product=2, SerialNumber=3
[89747.946582] usb 1-1: Product: CP2102 USB to UART Bridge Controller
[89747.946583] usb 1-1: Manufacturer: Silicon Labs
[89747.946584] usb 1-1: SerialNumber: TGLPR1Q6
[89747.962109] usbcore: registered new interface driver cp210x
[89747.962122] usbserial: USB Serial support registered for cp210x
[89747.962140] cp210x 1-1:1.0: cp210x converter detected
[89747.963587] usb 1-1: cp210x converter now attached to ttyUSB0
```



- Connect the module with the RaspberryPi



- Let's open **minicom** on the new USB serial port
- Let's start with configuring **minicom**

```
#> sudo minicom -s
```

```
+-----[configuration]-----+
| Filenames and paths          |
| File transfer protocols      |
| Serial port setup           |
| Modem and dialing           |
| Screen                       |
| Keyboard and Misc           |
| Save setup as dfl            |
| Save setup as..              |
| Exit                         |
| Exit from Minicom           |
+-----+

```

- Type **A** in the keyboard and change the path to **/dev/ttyUSB0**

```
+-----+
| A -   Serial Device       : /dev/ttyUSB0 |
| C -   Callin Program      :              |
| D -   Callout Program     :              |
| E -   Bps/Par/Bits        : 115200 8N1   |
| F -   Hardware Flow Control : No         |
| G -   Software Flow Control : No         |
| H -   RS485 Enable        : No           |
| I -   RS485 Rts On Send   : No           |
| J -   RS485 Rts After Send : No           |
| K -   RS485 Rx During Tx  : No           |
| L -   RS485 Terminate Bus : No           |
| M -   RS485 Delay Rts Before: 0          |
| N -   RS485 Delay Rts After : 0          |
|                                         |
|           Change which setting?          |
+-----+
```


- Make sure the baudrate is **115200** and flow control is disabled and press **Enter**

```
+-----+
| A -   Serial Device       : /dev/ttyUSB0
|
| C -   Callin Program      :
| D -   Callout Program     :
| E -   Bps/Par/Bits        : 115200 8N1
| F -   Hardware Flow Control : No
| G -   Software Flow Control : No
| H -   RS485 Enable        : No
| I -   RS485 Rts On Send   : No
| J -   RS485 Rts After Send : No
| K -   RS485 Rx During Tx  : No
| L -   RS485 Terminate Bus  : No
| M -   RS485 Delay Rts Before: 0
| N -   RS485 Delay Rts After : 0
|
|   Change which setting? █
+-----+
```

```
+-----[configuration]-----+
| Filenames and paths          |
| File transfer protocols      |
| Serial port setup           |
| Modem and dialing           |
| Screen                      |
| Keyboard and Misc           |
| Save setup as dfl           |
| Save setup as..             |
| Exit                        |
| Exit from Minicom           |
+-----+-----+-----+-----+
```

```
+-----[configuration]-----+
| Filenames and paths          |
| File transfer protocols      |
| Serial port setup            |
| Modem and dialing            |
| Screen                       |
| Keyboard and Misc            |
| Save setup as dfl            |
| Save setup as..              |
| Exit                         |
| Exit from Minicom            |
+-----+-----+
```

- Now, it is waiting for any incoming data, that data will come from the board when it boots up
- Now, let's power on the board with the USB cable and wait for the boot process
- The first boot will configure the board and then it will reboot automatically
- The second boot, it will show the login prompt

```
Welcome to minicom 2.9

OPTIONS: I18n
Compiled on Jan 17 2025, 00:00:00.
Port /dev/ttyUSB0, 21:08:06

Press CTRL-A Z for help on special keys
```



Press CTRL-A Z for help on special keys

```
[ 29.733322] reboot: Restarting system with command '1'
[ 18.801362] reboot: Restarting system
```

Debian GNU/Linux 12 **techleefrpi** ttyS0

My IP address is 192.168.1.127 fe80::733c:2db9:f5d0:7337

techleefrpi login: techleefrpi

Password:

Login incorrect

techleefrpi login: techleef

Password:

Linux techleefrpi 6.12.25+rpt-rpi-v8 #1 SMP PREEMPT Debian 1:6.12.25-1+rpt1 (2025-04-30) aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.

Last login: Tue May 13 01:18:11 BST 2025 on tty1

techleef@techleefrpi:~\$

- If you configure WiFi, the board should connect automatically

```
techleef@techleefrpi:~$ iw dev  
phy#0  
    Unnamed/non-netdev interface  
        wdev 0x2  
        addr [REDACTED]  
        type P2P-device  
        txpower 31.00 dBm  
Interface wlan0  
    ifindex 2  
    wdev 0x1  
    addr [REDACTED]  
    ssid Flybox-8A6F2E  
    type managed  
    channel 11 (2462 MHz), width: 20 MHz, center1: 2462 MHz  
    txpower 31.00 dBm
```

- If you configured a hostname and you are connected at the same wifi, you can ping the board

```
09:36:24 talel@techleef ~ → ping techleefrpi.local -c 3
PING techleefrpi.local (192.168.1.127) 56(84) bytes of data.
64 bytes from techleefrpi (192.168.1.127): icmp_seq=1 ttl=64 time=4.04 ms
64 bytes from techleefrpi (192.168.1.127): icmp_seq=2 ttl=64 time=7.40 ms
64 bytes from techleefrpi (192.168.1.127): icmp_seq=3 ttl=64 time=10.7 ms

--- techleefrpi.local ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 4.043/7.382/10.703/2.718 ms
09:36:29 talel@techleef ~ →
```

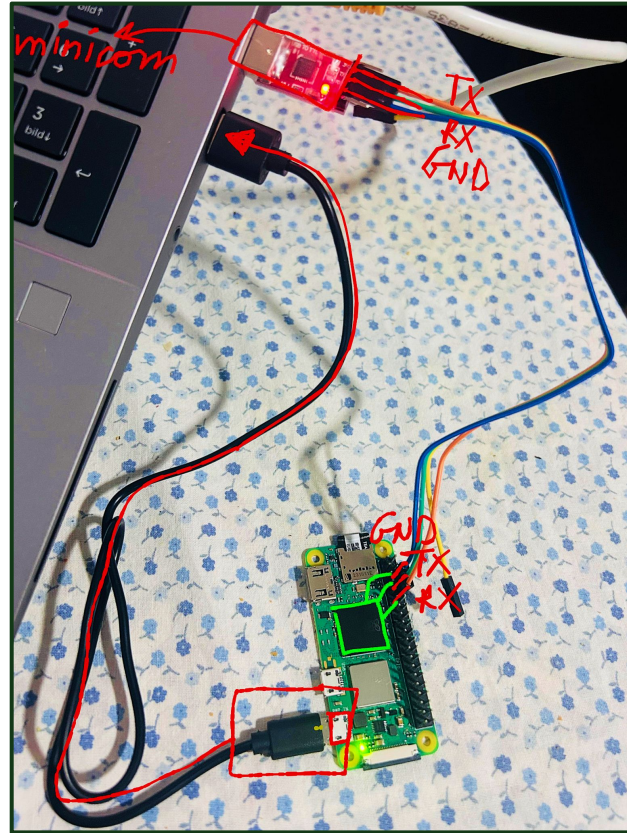
- If you enabled SSH, you can access the board via SSH

```
09:36:29 talel@techleef ~ → ssh techleef@techleefrpi.local
The authenticity of host 'techleefrpi.local (192.168.1.127)' can't be established.
ED25519 key fingerprint is SHA256:wheIFH0dr67rGzpREsQt5j/elExjGll7ctmDDDqTHys.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'techleefrpi.local' (ED25519) to the list of known hosts.
techleef@techleefrpi.local's password:
Linux techleefrpi 6.12.25+rpt-rpi-v8 #1 SMP PREEMPT Debian 1:6.12.25-1+rpt1 (2025-04-30) aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sat Sep 27 21:30:21 2025
techleef@techleefrpi:~ $
techleef@techleefrpi:~ $
techleef@techleefrpi:~ $ uname -a
Linux techleefrpi 6.12.25+rpt-rpi-v8 #1 SMP PREEMPT Debian 1:6.12.25-1+rpt1 (2025-04-30) aarch64 GNU/Linux
techleef@techleefrpi:~ $
```


Setup Example



Shell Package Manager

- We will start developing a debian package manager that is capable of
 - Listing information about installed packages
 - Listing information about a specific package
 - Install a package
 - Remove a package
 - Create a .deb package
- To have full control we will use several commands, essentially: **dpkg-query** and **apt**

- First thing to do when thinking about how to develop a script, is to learn more about the commands will be used
- The most important command for that goal is “**man**”, so let’s check the **dpkg-query** manual

```
#> man dpkg-query
```

```
dpkg-query(1)                                dpkg suite                                dpkg-query(1)
```

```
NAME
```

```
    dpkg-query - a tool to query the dpkg database
```

```
SYNOPSIS
```

```
    dpkg-query [option...] command
```

```
DESCRIPTION
```

```
    dpkg-query is a tool to show information about packages listed in the dpkg database.
```

```
..
```

- The most important parts we need to focus on are **COMMANDS** and **OPTIONS**

COMMANDS

`-l, --list [package-name-pattern...]`

List all known packages matching one or more patterns, regardless of their status, which includes any real or virtual package referenced in any dependency relationship field (such as Breaks, Enhances, etc.).

..

OPTIONS

`--admindir=dir`

Change the location of the dpkg database. The default location is `/var/lib/dpkg`.

..

- Let's extract the commands and optionally the options that we will need for our script design

Argument	Description
-L, --listfiles <package>...	List files 'owned' by package(s)
-l, --list [<pattern>...]	List packages concisely
-W, --show [<pattern>...]	Show information on package(s)
-f --showformat=<format>	Use alternative format for --show

- We start with the script shebang: **#!/bin/bash** telling the kernel the interpreter used for the script
- It MUST be the first line in the script

```
#!/bin/bash
```


- The first thing we need to do in the script is to check if **dpkg-query** command is available or not
- **&>** is used to redirect errors and non-error logs to same destination

```
if ! command -v dpkg-query &> /dev/null; then
    echo "[ERROR] dpkg-query is not installed, are you on a DEBIAN distro ?"
    exit 1
fi
```

- Now, let's develop the help function that we need to execute on demand and/or misuse of the script
- The help menu should show all available options, usually we name it “**usage**”
- We use **printf** command to print multiline message
- We define **exit_code** variable in case no argument is passed

```
usage(){  
    printf '%s\n' \  
    " Usage: ${0} info|install|remove|create|help ARGS" \  
    "" \  
    " - info      : show information of a given package" \  
    " - install   : install a given package" \  
    " - remove    : remove a given package" \  
    " - create    : create a .deb package from a given directory" \  
    " - help      : show this help"  
  
    local exit_code="${[ ${1} != 1 ] && echo 0 || echo 1}"  
    exit "${exit_code}"  
}
```

- Now, our script will accept arguments, so we need to control the number of arguments passed
- Our script can accept one argument and that could only be “**help**”
- Our script can accept 2 arguments for actions “**info**”, “**install**”, “**remove**” and “**create**”, plus the package name

```
# Number of arguments could be 1 or 2
if [ $# -eq 1 ]; then
    [ "${1}" != "help" ] && usage 1 || usage 0
elif [ $# -ne 2 ]; then
    usage 1
fi
```

- Now, if the arguments are checked with no error, we are sure that we have an action and an argument for it
- So, we can store them in global variables that we can use later

```
ACTION="${1}"  
ARGUMENT="${2}"
```

- Now, let's define the functions that we will use for each action

```
do_create(){  
    echo "[DEBUG] Creating a package for: ${1}"  
}  
  
do_install(){  
    echo "[DEBUG] Installing package: ${1}"  
}  
  
do_remove(){  
    echo "[DEBUG] Removing package: ${1}"  
}  
  
do_info(){  
    echo "[DEBUG] Showing info about package: ${1}"  
}
```

- Now, we need to map the action from command line argument to the corresponding function
- We can use Bash **associative array** feature (it is like Dictionaries in other programming languages)
- Using this mechanism we need to develop the **do_action** function now to call the required action

```
declare -A ACTIONS=(  
    [info]=do_info  
    [install]=do_install  
    [remove]=do_remove  
    [create]=do_create  
)  
  
do_action "${ACTION}" "${ARGUMENT}"
```

- Now, let's design the **do_action** function, it needs to do the following:
 - Check if the passed action exist in the associative array
 - We define a **cmd** variable holding the required function to call
- The following part is the check for action validation and execution

```
do_action(){  
    local action="${1}"  
    local package="${2}"  
    local cmd="${ACTIONS[${action}]}"  
  
    [ -z "${cmd}" ] && usage 1  
  
    echo "[DEBUG] Running command : ${cmd}"  
    eval "${cmd} ${package}"  
}
```

- Some actions required checking the package existence, (info, install, remove)
- We can develop a function for that

```
is_package_exist(){  
    dpkg-query -l "${package}" >/dev/null  
}
```


- Let's start developing our functions, starting with **do_info**
- **do_info** will list information about a given package
- To understand more about listing a package information, we can use the **-f|--showformat** option
 - This is to control the output information we can see
 - We can see the full supported list of format options in the **dpkg-query** manual
- We care about **Installed-Size**, **Package** and **Status**

`-f, --showformat=format`

This option is used to specify the format of the output `--show` will produce. The format is a string that will be output for each package listed.

Architecture

Bugs

Conflicts

Installed-Size

Package

Priority

Provides

Recommends

Status

- Let's implement that in **do_info** function
- It is **TOO IMPORTANT** to wrap the format options with single quote so the shell interpreter don't interpret **\$** signs

```
do_info(){
    local package="${1}"
    if is_package_exist "${package}"; then
        echo "[DEBUG] Showing info about package: ${package}"
        dpkg-query \
            --show \
            --showformat \
            'Package:${Package}\nSize:${Installed-Size}\nStatus:${db:Status-Status}\n' \
            ${package}
    else
        echo "[INFO] package: ${package} is not installed"
    fi
}
```

- Let's implement **do_remove**
- To remove a package, we can use “**apt**” or “**dpkg**”, for simplicity we will use **dpkg**

```
do_remove(){  
    local package="${1}"  
    if is_package_exist "${package}"; then  
        echo "[DEBUG] Removing package: ${package}"  
        dpkg -P ${package}  
    else  
        echo "[INFO] package: ${package} is already not installed"  
    fi  
}
```

- Let's implement **do_install**
- We can install a local package (**.deb**) or remote one
- Remote packages are installed with “**apt**”, also “**apt**” can be used to install local **.deb** packages
- But we need to detect, if the package name ends with “**.deb**” then it is local package

```
do_install(){  
    local package="${1}"  
  
    if [[ "${package}" == *.deb ]] && [ ! -f "${package}" ]; then  
        echo "[ERROR] Package file ${package} does not exist"  
        return  
    elif is_package_exist "${package}"; then  
        echo "[ERROR] Package ${package} is already installed"  
        return  
    fi  
  
    echo "[DEBUG] Installing package: ${package}"  
    apt install ${package}  
}
```

- Let's implement **do_create**
- To create a package, we need to prepare a **control** file containing package metadata information
- We need to create a destination folder structure same as where we want files to be in the rootfs
- The package naming should follow the recommended rule: **<name>_<version>_<architecture>.deb**
- If we will create a package for interpreted scripts (shell, python, ..) the architecture should be **all**

```
#> mkdir -p /tmp/techleef_0.1_all/{DEBIAN,usr/bin}
#> touch /tmp/techleef_0.1_all/DEBIAN/control
#> touch /tmp/techleef_0.1_all/usr/bin/techleef
#> chmod 755 /tmp/techleef_0.1_all/usr/bin/techleef
#> echo "echo -e Hello from TechLeef\n" >> /tmp/techleef_0.1_all/usr/bin/techleef
#> tree /tmp/techleef_0.1_all
```

```
/tmp/techleef_0.1_all/
├── DEBIAN
│   └── control
└── usr
    └── bin
        └── techleef
```

- We are going to use **dpkg-deb** command to create the package using the **-b** option
- Let's check the command manual and build the package

```
#> man dpkg-deb
```

COMMANDS

-b, --build binary-directory [archive|directory]

Creates a debian archive from the filesystem tree stored in binary-directory.

binary-directory **must have a DEBIAN subdirectory**, which contains the control information files such as the control file itself. This directory will not appear in the binary package's filesystem archive, but instead the files in it will be put in the binary package's control information area.

..

```
#> dpkg-deb -i techleef_0.1_all
```

```
dpkg-deb: building package 'techleef' in 'techleef_0.1_all.deb'.
```

- Let's continue implementing **do_create** now with **dpkg-deb**
- We need to make sure the given directory exists

```
do_create(){  
    local directory="${1}"  
    echo "[DEBUG] Creating a package for: ${directory}"  
    if [ ! -d "${directory}" ]; then  
        echo "[ERROR] Directory ${directory} does not exist"  
    else  
        dpkg-deb -b "${directory}"  
    fi  
}
```


- Let's install the created package, using the “**install**” action
- And then let's check the package info with “**info**” action
- Let's execute the **techleef** command

```
#> ./debmgr.sh create techleef_0.1_all
#> sudo ./debmgr.sh install techleef_0.1_all.deb
#> ./debmgr.sh info techleef
```

```
[DEBUG] Running command : do_info
[DEBUG] Showing info about package: techleef
Package : techleef
Size :
Status : installed
```

```
#> techleef
Hello from TechLeef
```

- When we remove the package, we can see there is no more **techleef** command

```
#> sudo ./debmgr.sh remove techleef

[DEBUG] Running command : do_remove
[DEBUG] Removing package: techleef
(Reading database ... 147209 files and directories currently installed.)
Removing techleef (1.0) ...

#> techleef

-bash: techleef: command not found
```

- Now, we will move on and check if our package manager helper is POSIX compliant
- Without even checking, Bash is not 100% POSIX compliant, it has extra non-POSIX features
- To check our script, we use the shellcheck utility
 - It has an online version, and package to install
 - The online version is not used with production code

```
#> sudo apt install shellcheck
```

- We will analyze the script as an “**sh**” script and not “**bash**”

Usage: shellcheck [OPTIONS...] FILES...

-a	--check-sourced	Include warnings from sourced files
-C[WHEN]	--color[=WHEN]	Use color (auto, always, never)
-i CODE1,CODE2..	--include=CODE1,CODE2..	Consider only given types of warnings
-e CODE1,CODE2..	--exclude=CODE1,CODE2..	Exclude types of warnings
	--extended-analysis=bool	Perform dataflow analysis (default true)
-f FORMAT	--format=FORMAT	Output format (checkstyle, diff, gcc, json, json1, quiet, tty)
	--list-optional	List checks disabled by default
	--norc	Don't look for .shellcheckrc files
	--rcfile=RCFILE	Prefer the specified configuration file over searching for one
-o check1,check2..	--enable=check1,check2..	List of optional checks to enable (or 'all')
-P SOURCEPATHS	--source-path=SOURCEPATHS	Specify path when looking for sourced files ("SCRIPTDIR" for script's dir)
-s SHELLNAME	--shell=SHELLNAME	Specify dialect (sh, bash, dash, ksh, busybox)
-S SEVERITY	--severity=SEVERITY	Minimum severity of errors to consider (error, warning, info, style)
-V	--version	Print version information
-W NUM	--wiki-link-count=NUM	The number of wiki links to show, when applicable
-x	--external-sources	Allow 'source' outside of FILES
	--help	Show this usage summary and exit

POSIX Programming

Inter Process Communication

Lab

we are going to develop a C socket server/client

Server Side

- We start by developing the server side
- The server needs to prepare the socket and then start listening on any incoming connection
- The socket type can be TCP, UDP or UNIX
- For a TCP socket, the server does the following:
 - Open a socket : **socket()**
 - Bind the socket with an IP and a port : **bind()**
 - Listen to incoming connections : **listen()**
 - Accept a connection in a dedicated client socket : **accept()**
 - Exchange with the client : **recv()** and **send()**
 - Close the client socket : **close()**
 - Close the server socket : **close()**

- The first step is opening a socket, let's check the manual for the “**socket**” function:

```
#> man 2 socket
```

NAME

socket - create an endpoint for communication

LIBRARY

Standard C library (libc, -lc)

SYNOPSIS

```
#include <sys/socket.h>
```

```
int socket(int domain, int type, int protocol);
```

DESCRIPTION

socket() creates an endpoint for communication and returns a file descriptor that refers to that endpoint. The file descriptor returned by a successful call will be the lowest-numbered file descriptor not currently open for the process.

- The first argument is the **domain**
- Linux supports lot of domains, most used domains are the following

DESCRIPTION

`socket()` creates an endpoint for communication and returns a file descriptor that refers to that endpoint. The file descriptor returned by a successful call will be the lowest-numbered file descriptor not currently open for the process.

The `domain` argument specifies a communication domain; These families are defined in `<sys/socket.h>`

Name	Purpose	Man page
AF_UNIX	Local communication	<code>unix(7)</code>
AF_INET	IPv4 Internet protocols	<code>ip(7)</code>
<code>AF_INET6</code>	IPv6 Internet protocols	<code>ipv6(7)</code>
<code>AF_CAN</code>	Controller Area Network automotive bus protocol	
<code>AF_BLUETOOTH</code>	Bluetooth low-level socket protocol	
..		

- The second argument is the **type**
- Most used types are **SOCK_STREAM** (TCP) and **SOCK_DGRAM** (UDP)
- The third argument is the protocol, which is usually 0

The socket has the indicated type, which specifies the communication semantics.

Currently defined types are:

- | | |
|--------------------|---|
| SOCK_STREAM | Provides sequenced, reliable, two-way, connection-based byte streams.
An out-of-band data transmission mechanism may be supported. |
| SOCK_DGRAM | Supports datagrams (connectionless, unreliable messages of a fixed maximum length). |
| .. | |

- Based on that, we can define a variable for the server socket, and open it

```
#include <stdio.h>
#include <sys/socket.h>

int main(int argc, char **argv){

    int server_fd;
    if((server_fd = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        perror("Cannot create TCP socket !");
        exit(1);
    }
}
```

- Next, we bind the socket to an IP and a port. Let's check the manual of the **bind** function

NAME

bind - bind a name to a socket

LIBRARY

Standard C library (libc, -lc)

SYNOPSIS

```
#include <sys/socket.h>
```

```
int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

DESCRIPTION

When a socket is created with `socket(2)`, it exists in a name space (address family) but has no address assigned to it. `bind()` assigns the address specified by **addr** to the socket referred to by the file descriptor **sockfd**. **addrlen** specifies the size, in bytes, of the address structure pointed to by **addr**. Traditionally, this operation is called “assigning a name to a socket”.

- The manual says that the “**addr**” argument’s `sockaddr` structure depends on the socket family
- In our case, the family is **AF_INET** and it is documented in entry **7** of the **ip** manual

The rules used in name binding vary between address families.

Consult the manual entries in Section 7 for detailed information.

for AF_INET, see ip(7);
for AF_INET6, see ipv6(7);
for AF_UNIX, see unix(7);
for AF_APPLETALK, see ddp(7);
for AF_PACKET, see packet(7);
for AF_X25, see x25(7);
and for AF_NETLINK, see netlink(7).

```
#> man 7 ip
```

```
struct sockaddr_in {
    sa_family_t    sin_family; /* address family: AF_INET */
    in_port_t      sin_port;   /* port in network byte order */
    struct in_addr sin_addr;   /* internet address */
};

/* Internet address */
struct in_addr {
    uint32_t        s_addr;    /* address in network byte order */
};
```


- Now, the bind function expects the addr argument's type to be: **struct sockaddr**
- But, **AF_INET** sockaddr struct is named: **struct sockaddr_in**
- So, we need to make sure to cast (**struct sockaddr_in**) to (**struct sockaddr**)
- So, we need to prepare the fields of **struct sockaddr_in**, let's check the (man 7 ip) documentation
- We will use **AF_INET** for **sin_family**, and **INADDR_ANY** for **in_addr.s_addr**

`sin_family` is always set to `AF_INET`.

`sin_port` contains the port in network byte order

`sin_addr` is the IP host address

`in_addr` should be assigned one of the `INADDR_*` values

`INADDR_LOOPBACK` (127.0.0.1) always refers to the local host via the loopback device;

`INADDR_ANY` (0.0.0.0) means any address for socket binding;

- For the port field **sin_port**, we need to convert the host short port (example: 2000) to network byte order
- For that, we use htons function, we can see the manual:

```
#> man htons
```

NAME

htonl, **htons**, ntohl, ntohs - convert values between host and network byte order

LIBRARY

Standard C library (libc, -lc)

SYNOPSIS

```
#include <arpa/inet.h>
```

```
uint16_t htons(uint16_t hostshort);
```

DESCRIPTION

The htons() function converts the unsigned short integer hostshort from host byte order to network byte order.

- Now, let's implement the bind function:

```
#include <arpa/inet.h>

int main(int argc, char **argv){
    // <...>

    struct sockaddr_in server_addr;

    server_addr.sin_family = AF_INET;
    server_addr.sin_addr.s_addr = INADDR_ANY;
    server_addr.sin_port = htons(2000);

    if(bind(server_fd, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
        perror("Failed to bind the socket!");
        close(server_fd);
        exit(1);
    }
}
```

- Now, let's listen for incoming connections

NAME

`listen` - listen for connections on a socket

LIBRARY

Standard C library (libc, -lc)

SYNOPSIS

```
#include <sys/socket.h>
```

```
int listen(int sockfd, int backlog);
```

DESCRIPTION

`listen()` marks the socket referred to by **sockfd** as a passive socket, that is, as a socket that will be used to accept incoming connection requests using `accept(2)`.

The **backlog** argument defines the **maximum length** to which the queue of pending connections for `sockfd` may grow.

- Let's implement listen function and listen for 1 incoming connection

```
// <...>

if(listen(server_fd, 1) < 0) {
    perror("Cannot listen for incoming connections");
    close(server_fd);
    exit(1);
}

printf("Server listening on 0.0.0.0:2000 ..\n");
```

- Now, let's accept connections

NAME

accept, accept4 - accept a connection on a socket

LIBRARY

Standard C library (libc, -lc)

SYNOPSIS

```
#include <sys/socket.h>
```

```
int accept(int sockfd, struct sockaddr *_Nullable restrict addr,  
           socklen_t *_Nullable restrict addrlen);
```

DESCRIPTION

The `accept()` system call is used with connection-based socket types (`SOCK_STREAM`, `SOCK_SEQPACKET`). It extracts the first connection request on the queue of pending connections for the listening socket, `sockfd`, creates a new connected socket, and returns a new file descriptor referring to that socket.

- Similar to the bind function, we need to cast client socket addr to (**struct sockaddr**)

```
// <...>

int client_fd;
struct sockaddr_in client_addr;
socklen_t client_addr_len = sizeof(client_addr);

client_fd = accept(server_fd, (struct sockaddr *)&client_addr, client_addr_len);
if (client_fd < 0) {
    perror("Cannot accept client connections");
    close(server_fd);
    exit(1);
}
```


- Now, let's start receiving something from the client and sends back something too
- Let's check the manual of the **recv** and **send** functions

NAME

`recv, recvfrom, recvmsg` - receive a message from a socket

LIBRARY

Standard C library (`libc`, `-lc`)

SYNOPSIS

```
#include <sys/socket.h>
```

```
ssize_t recv(int sockfd, void buf[.len], size_t len, int flags);
```

DESCRIPTION

The `recv()`, `recvfrom()`, and `recvmsg()` calls are used to receive messages from a socket. They may be used to receive data on both connectionless and connection-oriented sockets.

With a zero flags argument, `recv()` is generally equivalent to `read(2)` (but see NOTES). The `recv()` call is normally used only on a connected socket (see `connect(2)`).

NAME

send, sendto, sendmsg - send a message on a socket

LIBRARY

Standard C library (libc, -lc)

SYNOPSIS

```
#include <sys/socket.h>
```

```
ssize_t send(int sockfd, const void buf[.len], size_t len, int flags);
```

DESCRIPTION

The system calls send(), sendto(), and sendmsg() are used to transmit a message to another socket.

The send() call may be used only when the socket is in a connected state
With a zero flags argument, send() is equivalent to write(2)

- Let's define a buffer to receive from the client
- The **recv** function returns the number of bytes received, so the buffer needs to be terminated with that size
- We need another buffer to fill from stdin to send to the client

```
// <..>

char receive_buffer[1024];
char send_buffer[1024];

while(1) {

    int received_bytes = recv(client_fd, receive_buffer, sizeof(receive_buffer) - 1, 0);
    if(received_bytes <= 0) {
        printf("Client disconnected \n");
        break;
    }

    receive_buffer[received_bytes] = '\0';
    printf("Client: %s", buffer);
}
```

- After finishing everything, let's not forget to close all sockets

```
// <...>

while(1) {

    // <...>

    if (!fgets(send_buffer, sizeof(send_buffer), stdin))
        break;

    send(client_fd, send_buffer, strlen(send_buffer), 0);

}

close(server_fd);
close(client_fd);

return 0;

}
```

Client Side

- The client will basically use the same functions explained in the server side
- The client will basically:
 - Create a socket
 - Declare the server address information (**struct sockaddr_in**)
 - Connect to the server
 - Send and Receive data
 - Close the socket
- The only difference is that the **sin_addr** field of **sockaddr_in** needs to be created using **inet_pton**

SYNOPSIS

```
#include <arpa/inet.h>
```

```
int inet_pton(int af, const char *restrict src, void *restrict dst);
```

DESCRIPTION

This function converts the character string `src` into a network address structure in the `af` address family, then copies the network address structure to `dst`. The `af` argument must be either `AF_INET` or `AF_INET6`. `dst` is written in network byte order.

```
if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) { /* Handle error */ }

server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(port);

if (inet_pton(AF_INET, server_address, &server_addr.sin_addr) <= 0) { /* Handle error */ }
if (connect(sock, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) { /* Handle error */ }

while (1) {
    if (!fgets(send_buffer, sizeof(send_buffer), stdin)) { /* break */ }

    send(sock, message, strlen(message), 0);
    int bytes_received = recv(sock, buffer, sizeof(buffer) - 1, 0);
    if (bytes_received <= 0) { /* Server disconnected */ }

    buffer[bytes_received] = '\0';
    printf("Server: %s", buffer);
}
```

- Now, let's compile the client on the PC side
- And copy the server to the board and compile it there

```
#> gcc client.c -o client  
  
#> scp server.c techleef@techleefrpi.local:/home/techleef  
  
#> ssh techleef@techleefrpi.local  
  
board#> gcc server.c -o server
```


- Run the server on the board, and then the client on the PC side

```
board#> ./server
```

```
Server listening on 0.0.0.0:2000 ..
```

```
Hello from PC
```

```
Hello from RPI
```

```
#> ./client
```

```
Hello from PC
```

```
Hello from RPI
```

MultiThreading Server

- Our server cannot handle more than one client, and even it can, it cannot handle them in parallel
- We need to do multithreading to handle all clients at the same time
- Using multithreading, the while loop to send and receive can be in a separate thread for each client
- The idea is simple:
 - The server needs to keep accepting connections
 - For each incoming connection, create a thread to handle it

- Let's start by checking the function that creates a thread (**pthread_create**)

```
#> man pthread_create
```

LIBRARY

```
POSIX threads library (libpthread, -lpthread)
```

SYNOPSIS

```
#include <pthread.h>
```

```
int pthread_create(pthread_t *restrict thread,  
                   const pthread_attr_t *restrict attr,  
                   void *(*start_routine)(void *),  
                   void *restrict arg);
```

DESCRIPTION

The `pthread_create()` function starts a new thread in the calling process. The new thread starts execution by **invoking `start_routine()`**; `arg` is passed as the sole argument of `start_routine()`.

- The **pthread_create** function takes 4 arguments:
 - A pointer to **pthread_t** which is a typedef to unsigned long int (*bits/pthread_types.h*)
 - A constant pointer to **pthread_attr_t** which is a union used for advanced thread
 - It is used to have more control on the thread
 - Like, controlling the stack size, the detach behaviour, and more
 - This is not covered in this course
 - A pointer to a function that has the signature: **void * func (void * arg) {}**, this is the thread job to do
 - A pointer to the argument that will be passed to the function

- To handle a client, we need a socket (**int**) and socket address information (**struct sockaddr_in**)
- We can define a structure to hold them both

```
#include <pthread.h>

struct client {
    int socket;
    struct sockaddr_in addr;
};
```

- Now, we need to transform the way we accept connections
- The call to **accept** function needs to be inside the while loop
- If **accept** fails we simple break out of the iteration, we don't exit

```
while(1) {  
    if((client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &addr_len)) < 0){  
        perror("Cannot accept connections");  
        break;  
    }  
  
    struct client * client = (struct client *) malloc(sizeof(struct client));  
    client->socket = client_fd;  
    client->addr = client_addr;  
}
```

- Now, let's create a thread, **pthread_create** returns an int (0 on success)
- When **pthread_create** fails we need to close the client socket and move on to the next
- If it passes, we need to detach from the thread and move on to accept another client
- To detach a thread, we need to use **pthread_detach**
- We can chose to do not detach the thread, but detaching removes the responsibility to clean it


```
while(1) {  
    // ..  
  
    pthread_t thread_id;  
  
    if(pthread_create(&thread_id, NULL, handle, client) != 0) {  
  
        // fail to create the thread  
        perror("Failed to create a thread");  
        close(client_fd);  
        free(client);  
  
    } else {  
        // success  
        pthread_detach(thread_id);  
    }  
}
```

- Now, we need to develop the handle function that will be responsible for send/recv
- It accepts a **void *** argument, which is the passed struct client
 - So, we need to cast it
- We need to move the send and receive buffers inside the function
- Then, we keep looping and trying to receive and send something

```
void * handle(void * arg) {  
  
    struct client * client = (struct client *) arg;  
  
    char send_buffer[1024];  
    char recv_buffer[1024];  
  
    printf("A client connected: %s:%d\n",  
          inet_ntoa(client->addr.sin_addr),  
          ntohs(client->addr.sin_port));  
  
    while (1) {  
        // same recv and send block  
    }  
}
```

- After getting out of the while true, we need to exit the thread
- Since the thread is detached, we need to use **pthread_exit**

NAME

`pthread_exit` - terminate calling thread

LIBRARY

POSIX threads library (libpthread, -lpthread)

SYNOPSIS

```
#include <pthread.h>
```

```
[[noreturn]] void pthread_exit(void *retval);
```

DESCRIPTION

The `pthread_exit()` function terminates the calling thread and returns a value via `retval` that (if the thread is joinable) is available to another thread in the same process that calls `pthread_join(3)`.

```
void * handle(void * arg) {  
  
    // ..  
  
    while (1) {  
        // ..  
    }  
  
    pthread_exit(NULL);  
}
```

Next Steps

Step 1: Try to transform the server to broadcast all messages to all clients (group chat mode)

Step 2: You can work on enabling typing and receiving messages at the same time

Step 3: Use multiprocessing instead of multithreading

Step 4: Work on other IPC mechanisms (shared memory, file sharing, other types of sockets (unix, UDP))

Step 5: Work on other POSIX topics (signal handling, file handling, process management)